Project 1: Image Processing

Implementation

This program reads image tasks from effects.txt and applies each list of effects in order. It supports grayscale, sharpen, blur, and edge detection. The last three use 3 by 3 convolution, with zero padding at the edges. Each effect reads from one buffer and writes to the other. Then the buffers are swapped, so there is no need to copy all the pixels.

The sequential version handles one image at a time. In parfiles, workers take images from a shared queue protected by a self-written TAS lock. They release the lock before processing an image. A WaitGroup waits for all workers to finish. There are no more workers than image tasks.

In parslices, workers split the rows of one image. Each worker writes only its own rows. After each effect, a WaitGroup waits for all rows to finish before swapping the buffers. Loading and saving still happen one image at a time.

All versions use draw.Draw to convert images to RGBA64 and BestSpeed to save PNGs. BestSpeed keeps the pixels unchanged but may produce larger files. All three versions use the same settings.

How to run

Put the supplied data directory inside proj1. From the project folder, run:

```
cd proj1/editor
go run editor.go small
go run editor.go small parfiles 4
go run editor.go small parslices 4
```

On Peanut, use Go 1.19.13 in $HOME/go. Install matplotlib if needed, then submit the experiment:

```
python3 -m pip install --user matplotlib
cd proj1/benchmark
sbatch benchmark-proj1.sh
```

The script runs the experiments and saves the results in results-cluster. run.py records the times, and analyze.py calculates speedups and draws the two graphs. A small timing program runs the same code as editor.go and records more decimal places. Timing includes loading, filtering, and saving, but not compiling or starting the program.

Experiment setup

I used Peanut's peanut-cpu partition, node p001, job 2615879, with go version go1.19.13 linux/amd64. The job had 16 CPUs. GOMAXPROCS was 1 for sequential runs and 16 for parallel runs.

I ran small, mixture, and big separately, using 2, 4, 6, 8, and 12 workers for each parallel mode. Each setting ran five times, including the sequential version: 165 runs total. I used the shortest time for each setting. Each dataset has ten images, so parfiles uses only ten workers when twelve are requested. Repeated runs may be faster because files are already cached.

Sequential times and Amdahl's law

All times are in seconds. Load, filter, and save times are from the run with the shortest total time.

Dataset	Total T1	Load	Filters	Save
small	2.347979	0.363852	0.896209	1.087604
mixture	14.312596	2.302493	5.402753	6.607101
big	35.211518	5.259305	14.316786	15.635188

For parfiles, P = load + filters + save. For parslices, P = filters. The parallel fraction is p = P / T1. Amdahl predicts S(n) = 1 / ((1 - p) + p/n), with a limit of S(infinity) = 1 / (1 - p). The next page compares the predictions with the measured speedups.

Dataset	Mode	P (seconds)	p	S(infinity)
small	parfiles	2.347665	0.999866	7465.35
small	parslices	0.896209	0.381694	1.62
mixture	parfiles	14.312347	0.999983	57490.00
mixture	parslices	5.402753	0.377482	1.61
big	parfiles	35.211279	0.999993	147144.44
big	parslices	14.316786	0.406594	1.69

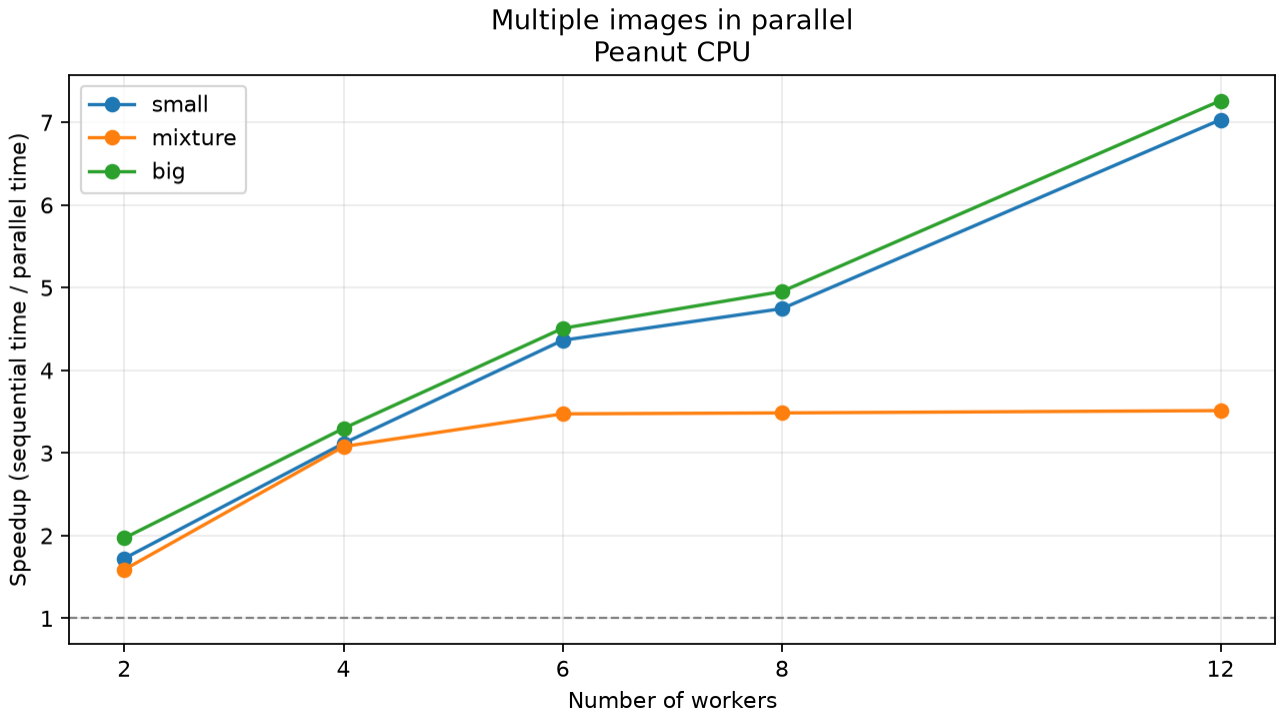
Amdahl assumes the parallel work splits perfectly. Real workers also spend time starting and waiting. The huge parfiles limit is not realistic here: there are only ten images, and everyone must wait for the last image to finish.

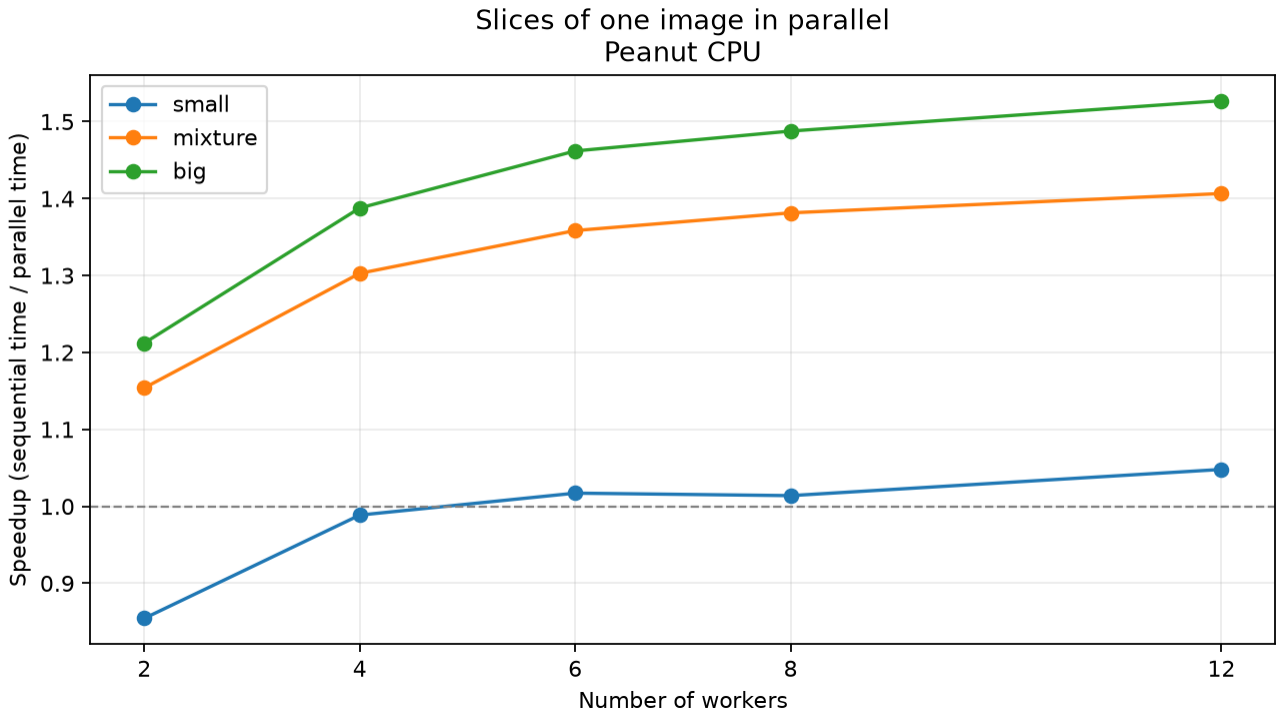
Measured and predicted speedup

Best time means the shortest of five runs. Speedup = sequential time / parallel time. Numbers are rounded here; the CSV files keep the full values.

Dataset	Mode	Workers	Best time (s)	Speedup	Amdahl
small	parfiles	2	1.3614	1.725	2.000
small	parfiles	4	0.7533	3.117	3.998
small	parfiles	6	0.5380	4.364	5.996
small	parfiles	8	0.4947	4.747	7.993
small	parfiles	12	0.3339	7.032	11.982
small	parslices	2	2.7480	0.854	1.236
small	parslices	4	2.3749	0.989	1.401
small	parslices	6	2.3084	1.017	1.466
small	parslices	8	2.3158	1.014	1.501
small	parslices	12	2.2405	1.048	1.538
mixture	parfiles	2	9.0054	1.589	2.000
mixture	parfiles	4	4.6491	3.079	4.000
mixture	parfiles	6	4.1200	3.474	5.999
mixture	parfiles	8	4.1068	3.485	7.999
mixture	parfiles	12	4.0739	3.513	11.998
mixture	parslices	2	12.4029	1.154	1.233
mixture	parslices	4	10.9860	1.303	1.395
mixture	parslices	6	10.5360	1.358	1.459
mixture	parslices	8	10.3607	1.381	1.493
mixture	parslices	12	10.1756	1.407	1.529
big	parfiles	2	17.8686	1.971	2.000
big	parfiles	4	10.6781	3.298	4.000
big	parfiles	6	7.8095	4.509	6.000
big	parfiles	8	7.1068	4.955	8.000
big	parfiles	12	4.8482	7.263	11.999
big	parslices	2	29.0465	1.212	1.255
big	parslices	4	25.3729	1.388	1.439
big	parslices	6	24.0867	1.462	1.512
big	parslices	8	23.6678	1.488	1.552
big	parslices	12	23.0553	1.527	1.594

Speedup graphs





Results and bottlenecks

On small, parfiles reached 7.03x speedup at 12 requested workers, while parslices reached 1.05x at 12. Comparing their best results, parfiles was faster. The slowest sequential stage was save, at 46.3% of total time.

On mixture, parfiles reached 3.51x speedup at 12 requested workers, while parslices reached 1.41x at 12. Comparing their best results, parfiles was faster. The slowest sequential stage was save, at 46.2% of total time.

On big, parfiles reached 7.26x speedup at 12 requested workers, while parslices reached 1.53x at 12. Comparing their best results, parfiles was faster. The slowest sequential stage was save, at 44.4% of total time.

Parfiles does better because workers can load, filter, and save different images at the same time. Parslices only speeds up filtering. Loading and saving still limit its total speedup. These timings show which stage is slowest, not which individual function uses the most CPU time.

For parfiles, best speedup rose from 7.03x on small to 7.26x on big. Larger images give workers more useful work compared with the time spent starting and waiting.

For parslices, best speedup rose from 1.05x on small to 1.53x on big. Larger images give workers more useful work compared with the time spent starting and waiting.

Two slice workers on small took 2.748 seconds, slower than the sequential time of 2.348 seconds. Starting workers and waiting at barriers may cost more than the time saved on a small image. I did not measure these costs separately.

For mixture, parfiles barely improves after six workers: 3.47x at six versus 3.51x at twelve. Small images may finish early, leaving a few large ones. Idle workers cannot help with an image already assigned to another worker.

Measured speedups were 29% to 99% of the Amdahl predictions. Starting workers, waiting at barriers, and uneven image sizes can reduce speedup. Workers also share memory and file access. At twelve requested workers, parfiles is still limited to ten image tasks.

Possible improvements

I could reuse slice workers between effects to avoid starting new ones each time. Another idea is to split a large remaining image among idle workers, while limiting the total number of workers. These ideas would need new experiments. Improving filters alone cannot remove the time spent loading and saving.